



Introduction to Statistics in R

Workshop 1 – September 22, 2026





Data exploration and visualization in R

Hands-on tutorials to help build skills required in Precision Child Health, data science and bioinformatics for SickKids staff and trainees.

Topics include:

- Data exploration in R
- Preparing data for analysis
- Data visualization using ggplot

No prior knowledge of programming is required. We encourage you to bring your laptop to practice data analysis steps under our guidance.

To register please visit:

<http://ccm20260922.eventbrite.com/>

Hands-on Data Science Tutorials for Biologists

Tuesday Sep 22, 2026
12:00 – 1:00 pm

Multi-media Room
PGCRL 3rd floor
or online (hybrid event)



Supported by :

Centre for Computational Medicine (ccm.sickkids.ca)

Digital Research Alliance of Canada (alliancecan.ca)

Compute Ontario (computeontario.ca)

CCM Overview

Note that prior knowledge of R is recommended to get the most out of this workshop series.

Workshop 1	Data exploration and visualization
Workshop 2	Statistical tests and models in R

Check out other workshop series hosted by CCM at <https://ccm.sickkids.ca/bioinformatics-training/>



Table of contents

01

Data exploration and cleaning

Overview of the Framingham heart dataset

Data exploration

Basic data frame manipulation

02

Data visualization

Plotting with base R vs ggplot2.

Distributions

Bar charts

Scatterplots

Panel figures





01

Data exploration and cleaning

Framingham Heart Study

One of the longest prospective epidemiological studies of cardiovascular disease and its risk factors.

Began in 1948 in Framingham, MA.

- Initially enrolled 5209 men and women aged 29–62 years old.
- Followed them over time, with assessments every 2 years.
- Data collection for the original cohort ended in 2014.
- Over time, the researchers incorporated offspring and their spouses into the study.
- Today, they have data from three generations of participants as well as more ethnically diverse cohorts.

Framingham Heart Study

Examinations included:

- Detailed medical history
- Physical exams
- Lab tests
- Lifestyle and habits
- Noninvasive imaging

Our dataset: subset of the FHS from Kaggle

- 4000+ records and 16 variables.

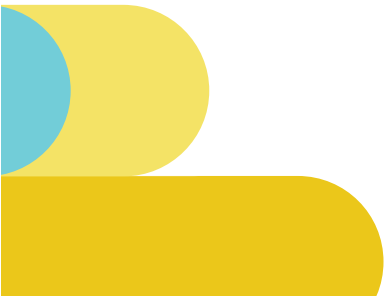
<https://www.kaggle.com/datasets/captainozlem/framingham-chd-preprocessed-data>

Variable	Description	Class/type
male	Sex (male or female)	Binary
age	Age	Continuous
education	Education (0-11 years, HS or GED, some uni, uni grad+)	Ordinal/categorical
currentSmoker	Whether the patient is currently a smoker	Binary
cigsPerDay	Number of cigarettes smoked per day, on average	Continuous
BPMeds	Whether the patient is on blood pressure medication	Binary
prevalentStroke	Whether the patient previously had a stroke	Binary
prevalentHyp	Whether the patient is hypertensive	Binary
diabetes	Whether the patient has diabetes	Binary
totChol	Total cholesterol level	Continuous
sysBP	Systolic blood pressure	Continuous
diaBP	Diastolic blood pressure	Continuous
BMI	Body mass index	Continuous
heartRate	Heart rate	Continuous
glucose	Glucose level	Continuous
TenYearCHD	10-year risk of coronary heart disease	Binary

Data exploration

Some common and/or important data cleaning steps include making sure that:

1. All variables are in the right format and/or correctly classified.
2. Duplicate or missing data are dealt with.
3. Variables of interest have been transformed appropriately.
4. Outliers are handled.
5. Data are validated.



Data exploration

1. Look at the data and its structure

```
str(df)
```

```
names(df)
```

```
nrow(df)/ncol(df)
```

```
head(df, n = X)/tail(df, n = X)
```

2. Tabulate data

```
table(df$column) or table(df$column1, df$column2, ...)
```

3. Modify variables

```
mutate()
```



Data exploration

4. Subset dataframes

`filter()` rows according to some condition

`select()` columns to keep

`subset(df, subset = row_vals == "abc", cols = X)`

5. Combine vectors and dataframes

`cbind()` to join by columns

`rbind()` to join by rows

`merge()` to join dataframes

6. Reshape dataframes

`pivot_wider()` each row is a participant/patient

`pivot_longer()` each row is an observation, with multiple rows per participant

Data exploration

7. Handy functions, pipes, and operators

`ifelse()` for conditional element selection

`case_when()` as a vectorized version of multiple `ifelse()` statements, uses formula notation (more on that later)

`%>%` passes the result of the function on the left as input to the next function

`%in%` to compare vectors

Create a summary table

Use `summarize()` to create a dataframe with summaries of desired variables. For example, what if you're only interested in the educational attainment and smoking habits of people who go on to develop heart disease?

	TenYearCHD	education_class	n_participants	age	currentSmoker	prop_smokers	cigsPerDay
1	0	< High School	1397	51	645	0.46	8.39
2	0	College Graduate	403	47	204	0.51	9.09
3	0	High School Graduate	1106	47	598	0.54	9.67
4	0	Some College	599	48	275	0.46	7.70
5	1	< High School	323	56	160	0.50	9.93
6	1	College Graduate	70	53	36	0.51	11.97
7	1	High School Graduate	147	52	81	0.55	11.14
8	1	Some College	88	53	46	0.52	10.74



02

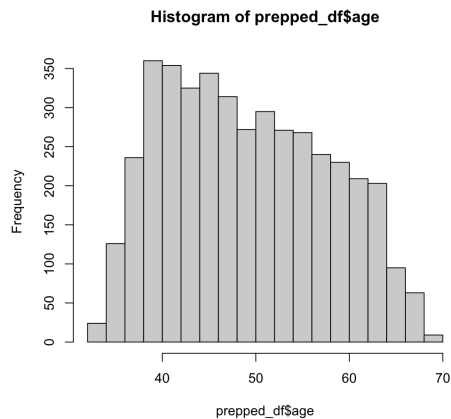
Data visualization

Base R vs ggplot2

Base R

Useful in a pinch but doesn't generate the prettiest figures.

`plot()` and `hist()` are useful for taking a quick look at your data.



ggplot2

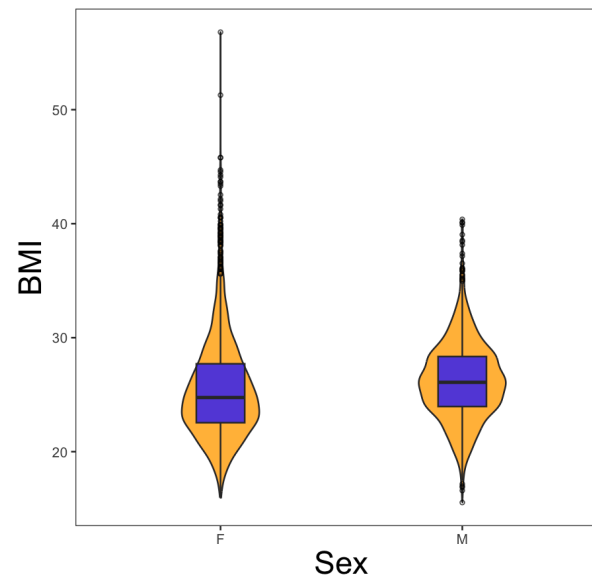
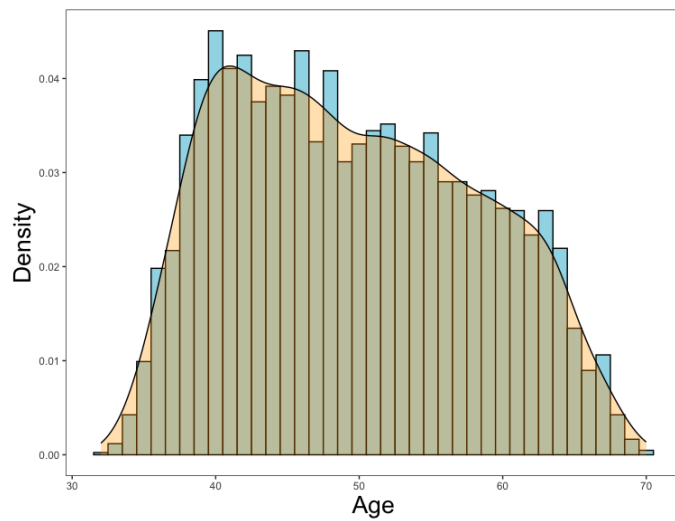
ggplot2 is a package based on the grammar of graphics.

You can create your figure with some basic components:

1. **Data**
2. **Mapping**
3. **Layers**
4. Scales
5. Facets
6. Coordinates
7. Theme

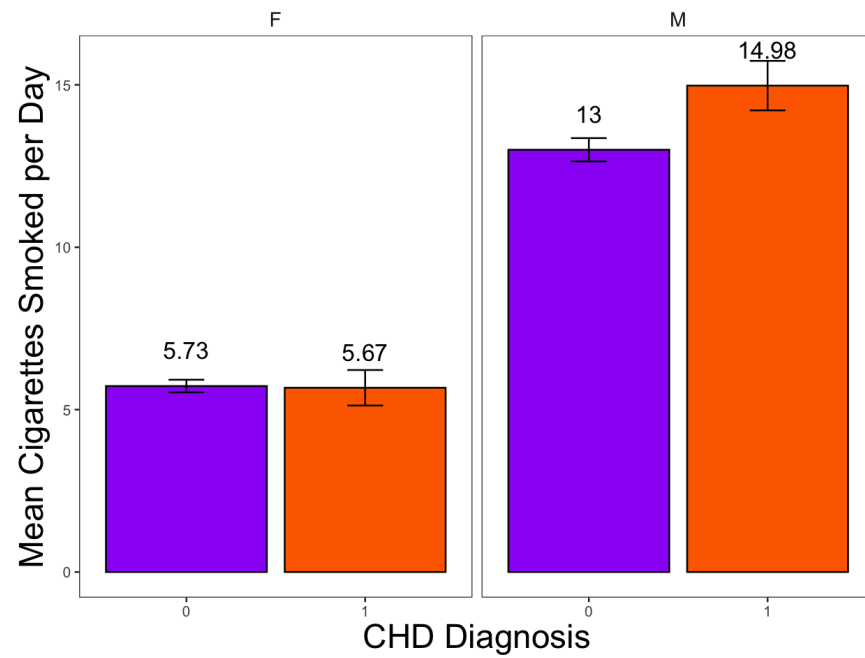
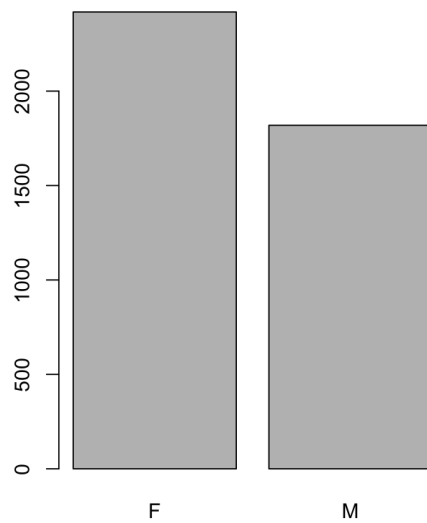
Distributions

Look at the distribution of data using histograms, density plots, boxplots, and violin plots (among others).



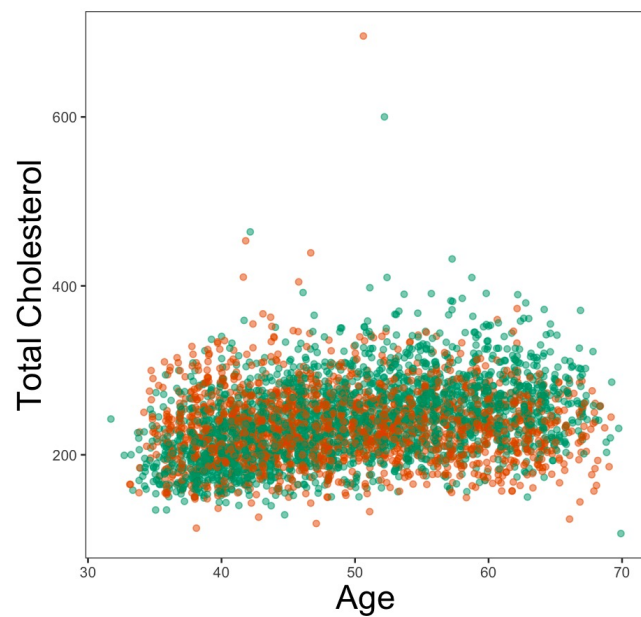
Bar Charts

Compare groups using bar charts.

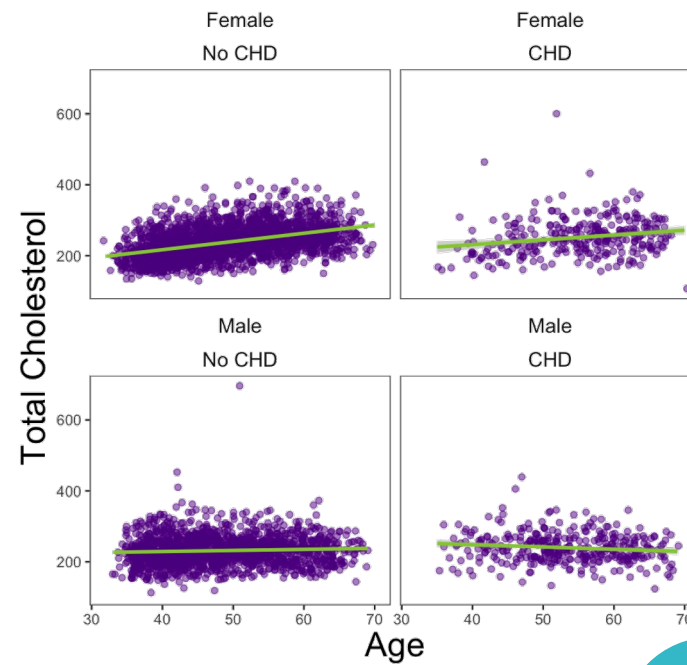


Scatterplots

Plot the relationship between two continuous variables using scatterplots.

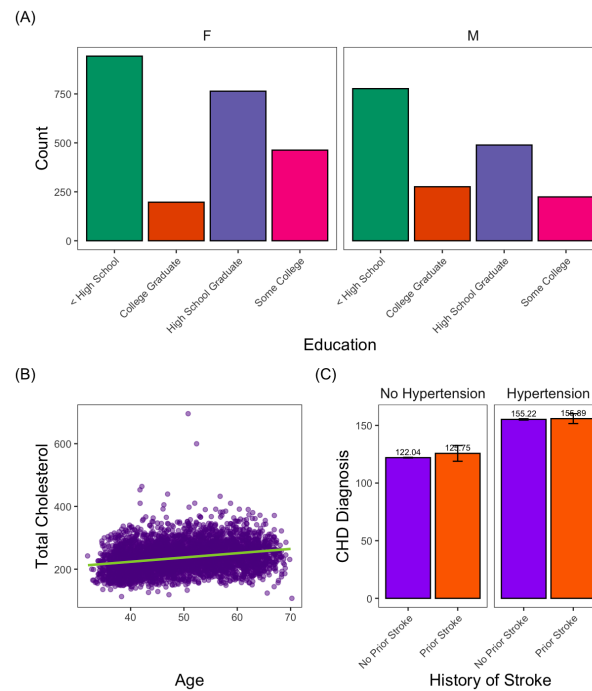


F
M



Panel figures

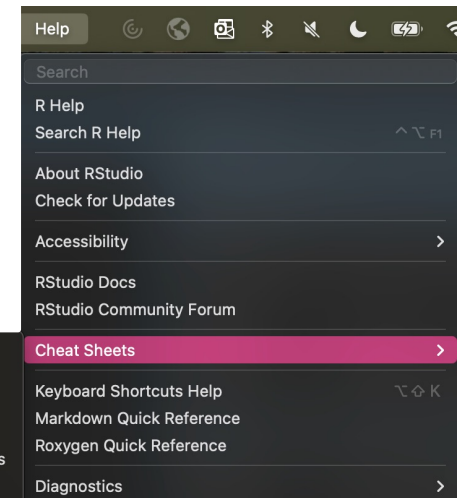
You might have multiple plots that you want to put into a single figure with panels. You can do this using the `patchwork()` package.



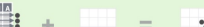
<https://www.maths.usyd.edu.au/u/UG/SM/STAT3022/r/current/Misc/data-visualization-2.1.pdf>

<https://sape.inf.usi.ch/quick-reference/ggplot2/colour>

- RStudio IDE Cheat Sheet
- Data Transformation with dplyr
- Data Visualization with ggplot2
- List manipulation with purrr
- Package Development with devtools
- Web Applications with shiny
- Interfacing Spark with sparklyr
- R Markdown Cheat Sheet
- R Markdown Reference Guide



ggplot2 is based on the **grammar of graphics**, the idea that you can build every graph from the same components: a **data set**, a **coordinate system**, and **geoms**—visual marks that represent data points.



The diagram illustrates the grammar of graphics, showing how data is transformed into a plot. It consists of four main components connected by mathematical symbols:

- data**: A table with columns `x` and `y`, and rows of data points.
- geom**: A plus sign (+) indicating the addition of a geometric layer.
- coordinate system**: A grid representing the coordinate system.
- plot**: The final result, a scatter plot where points are colored by a third variable `z`.

The equation is: **data** + **geom** = **coordinate system** = **plot**.

To display values, map variables in the data to visual properties of the geom (**aesthetics**) like **size**, **color**, and **x** and **y** locations.

Use a geom function to represent data points, use the geom's aesthetic properties to represent variables. Each function returns a layer.

```
a <- ggplot(economics, aes(date, unemploy))
b <- ggplot(seals, aes(x = long, y = lat))
```

-  **a + geom_blank()**
(Useful for expanding limits)
-  **b + geom_curve(aes(yend = lat + 1, xend=long+curvature/2))** x = xend, y = yend, alpha, angle, color, curvature, linetype, size
-  **a + geom_path(lineend="butt", linejoin="round", linemitre=1)**
x, y, alpha, color, group, linetype, size
-  **a + geom_polygon(aes(group = group))**
x, y, alpha, color, fill, group, linetype, size
-  **b + geom_rect(aes(xmin = long, ymin=lat, xmax=x + 1, ymax = lat + 1))** xmax, xmin, ymax, ymin, alpha, color, fill, linetype, size
-  **a + geom_ribbon(aes(ymin=unemployment - 900, ymax=unemployment + 900))** x, ymax, ymin, alpha, color, fill, group, linetype, size

```
continuous x , continuous y
a <- ggplot(mpg, aes(cty, hwy))
```

- e geom_label(aes(label = cty), nudge_x = 1, nudge_y = 1, check_overlap = TRUE), x, y, label, alpha, color, family, fontface, hjust, linewidth, size, vjust**
- e geom_jitter(hedge = 2, width = 2) x, y, alpha, color, fill, shape, size**
- e geom_point(), x, y, alpha, color, fill, shape, size, stroke**
- e geom_quantile(), x, y, alpha, color, fill, linetype, size, weight**
- e geom_rug(sides = "bf"), x, y, alpha, color, linewidth, size**
- e geom_smooth(method = lm), x, y, alpha, color, fill, group, linetype, size, weight**
- e geom_text(aes(label = cty), nudge_x = 1, nudge_y = 1, check_overlap = TRUE), x, y, label, alpha, color, family, fontface, hjust, linewidth, size, vjust**

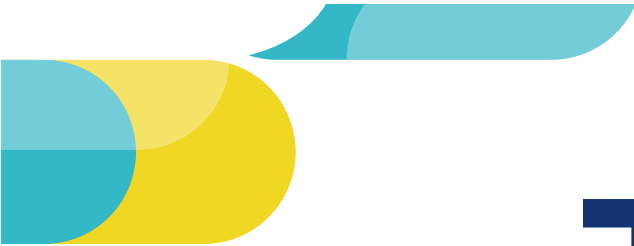
```
h + geom_bin2d(binwidth = c(0.25, 500))
```

- h + geom_binwidth()**
x, y, alpha, colour, fill, linetype, size, weight
- h + geom_density2d()**
x, y, alpha, colour, group, linetype, size
- h + geom_hex()**
x, y, alpha, colour, fill, size

```
i <- ggplot(economics, aes(date, unemploy))
```

- `i + geom_area()`
x, y, alpha, color, fill, linetype, size
- `i + geom_line()`
x, y, alpha, color, group, linetype, size
- `i + geom_step(direction = "hv")`
x, y, alpha, color, group, linetype, size





Thanks!

Do you have any questions?

shireen.parimoo@sickkids.ca



CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)

